## SIMATS ENGINEERING

# ASSESSMENT TOOL 1 — ANSWER KEY

Course: Object Oriented Analysis and Design

Course Code: CSA11

*Course Outcome: CO1 — Apply object-oriented concepts and software development life cycle models to design structured and modular software systems (BL2, BL3)*

**Short Answer Test • 6 Questions • Total Marks: 30**

## Question 1 — Library Management System

> *Suppose a software system is developed for a Library Management System where users can search books, borrow, and return them.*

### (a) Fundamental object-oriented concepts

- **Class:** A class is a blueprint or template that defines the structure (attributes) and behaviour (methods) of a set of objects, without occupying memory itself. In the LMS, Book and User are classes — Book defines attributes like title, author and isbn, while User defines userId and name.
- **Object:** An object is a concrete instance of a class, created at runtime and occupying memory. For example, "Book1: Python101" is an object of the Book class, holding its own specific attribute values.
- **Encapsulation:** Encapsulation binds data and the methods that operate on it into a single unit, and hides the internal state from outside interference by marking attributes private and exposing controlled access through public methods. In the LMS, a book's availability flag is kept private and can only be read or changed through a method such as checkStatus(), preventing other parts of the system from corrupting it directly.
- **Inheritance:** Inheritance allows a new class to reuse the attributes and methods of an existing class, forming an "is-a" relationship and promoting code reuse. In the LMS, Student and Staff can inherit common fields (userId, name) and behaviours (login(), logout()) from a base User class, while adding their own specialised attributes.
- **Polymorphism:** Polymorphism allows the same method name to exhibit different behaviour depending on the object that invokes it. For example, both Student and Staff classes can override a borrowBook() method inherited from User, each implementing its own borrowing rules (e.g., different loan durations), while calling code can invoke borrowBook() uniformly without knowing the exact subclass.

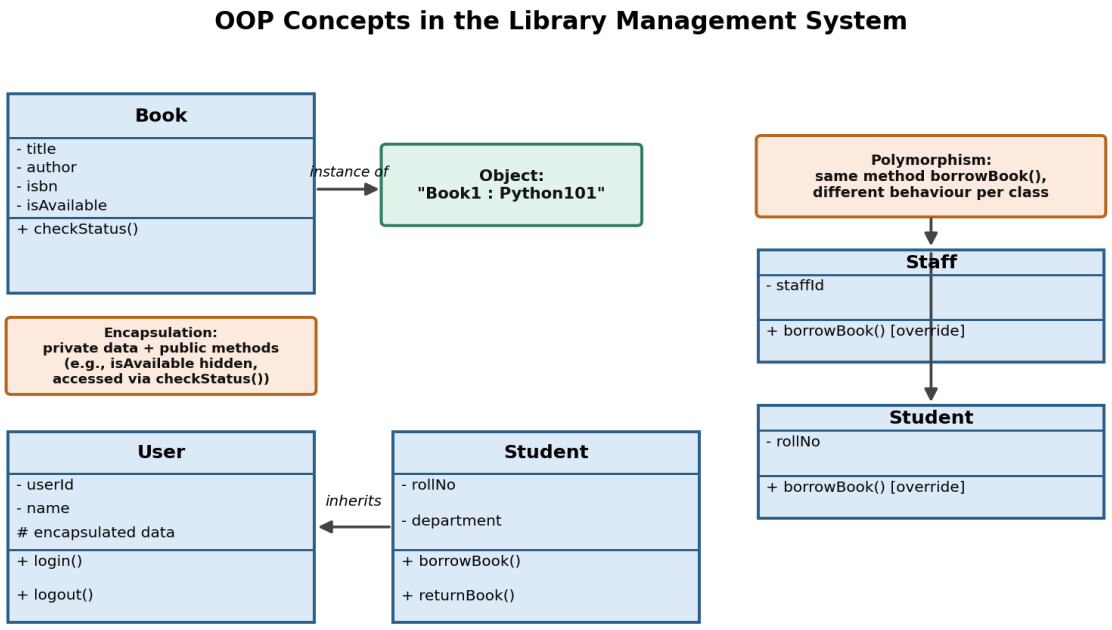


*Fig 1.1 — Class, Object, Encapsulation, Inheritance and Polymorphism in the LMS*

### (b) Object relationships

- **Association:** A general, loosely-coupled connection between two independent classes, where each can exist without the other. Example: a User is associated with a Book when they borrow it — the relationship exists only for the duration of the transaction.
- **Aggregation:** A special "has-a" relationship representing whole-part ownership where the part can exist independently of the whole (a weak form of composition). Example: a Library "has" Book objects, but a Book can still exist as a catalogued entity even if removed from that particular library branch.
- **Composition:** A stronger form of aggregation where the part cannot exist without the whole — if the whole is destroyed, its parts are destroyed too. Example: a LibraryCard object owns a Barcode object; the barcode has no meaning or existence outside its card.

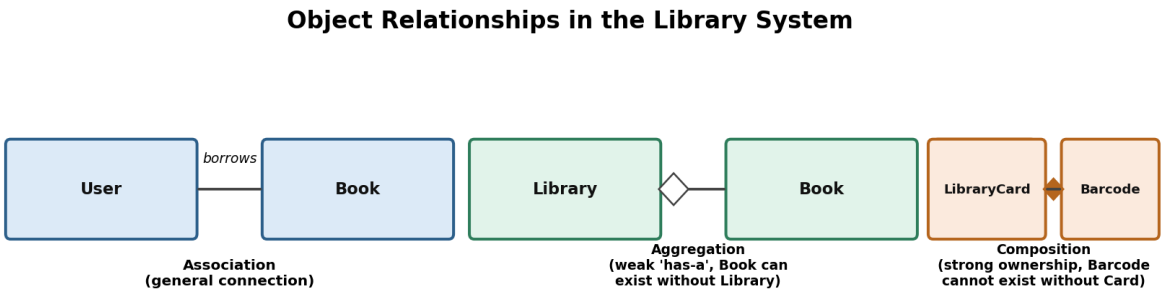


*Fig 1.2 — Association, Aggregation and Composition*

### (c) How these concepts build a modular, reusable system

- Encapsulation hides internal implementation, so classes like Book or User can be modified internally without breaking other parts of the system — improving maintainability.
- Inheritance lets common behaviour (e.g., login/logout) be written once in a base class and reused across Student, Staff and Admin — reducing duplication and improving reusability.
- Polymorphism allows new user types to be added later (e.g., Guest) simply by implementing the expected methods, without changing existing calling code — improving flexibility.
- Clear class boundaries and relationships (association, aggregation, composition) let the system be built as independent, pluggable modules (search module, borrowing module, catalogue module), which improves scalability and makes testing and future upgrades easier.

## Question 2

# Question 2 — Student Information System

> *Consider developing a Student Information System.*

### (a) Key object-oriented design principles

- **Abstraction:** Abstraction means exposing only the essential features of an entity while hiding unnecessary implementation detail from the user. In the Student Information System, a user calling

enrollStudent() does not need to know how records are stored internally — only that the operation enrols the student.

- **Encapsulation:** Encapsulation protects an object's internal data from unintended external access or modification by bundling data with the methods that operate on it and restricting direct access, e.g. a student's marks are private and only updated through an updateMarks() method that can validate the input.
- **Modularity:** Modularity is the practice of dividing a large system into smaller, independent, self-contained components (modules) such as Enrollment, Attendance, Examination and Fees, each of which can be developed, tested and maintained separately.

### (b) SRP and OCP

- **Single Responsibility Principle (SRP):** A class should have only one reason to change — i.e., one clearly defined responsibility. For example, a Student class should manage only student data, while report generation should be handled by a separate ReportGenerator class, not mixed into Student.
- **Open/Closed Principle (OCP):** Software entities should be open for extension but closed for modification. New functionality should be added by extending existing classes (e.g., through inheritance or interfaces) rather than editing already-tested code. For example, adding a new FeeDiscountPolicy should not require changing the existing Fee class, only adding a new class that implements a common interface.

### (c) How these principles improve software quality

- Reduce complexity by keeping each class focused and easy to understand.
- Improve code reuse, since well-abstracted, single-purpose modules can be reused across features.
- Improve maintainability, because changes are localised to one module/class instead of rippling across the system.
- Lower the risk of introducing bugs when extending the system, since OCP encourages adding new code rather than editing proven code.

## Question 3

## Question 3 — Online Shopping System (SDLC Phases)

> *Suppose a company plans to develop an Online Shopping System.*

### (a) Phases of SDLC

- **1. Requirement Analysis:** Gathering and documenting functional needs (product catalogue, cart, payment) and non-functional needs (performance, security) from stakeholders.
- **2. Design:** Translating requirements into system architecture, database schema, and UI/UX design, including high-level and detailed design documents.
- **3. Implementation:** Writing the actual code for modules such as product listing, shopping cart, checkout and payment gateway integration.

- **4. Testing:** Verifying the system through unit, integration, system and user-acceptance testing to detect and fix defects.
- **5. Deployment:** Releasing the tested system to the live/production environment for real customers to use.
- **6. Maintenance:** Ongoing bug fixes, performance tuning, and feature enhancements after deployment, based on user feedback.

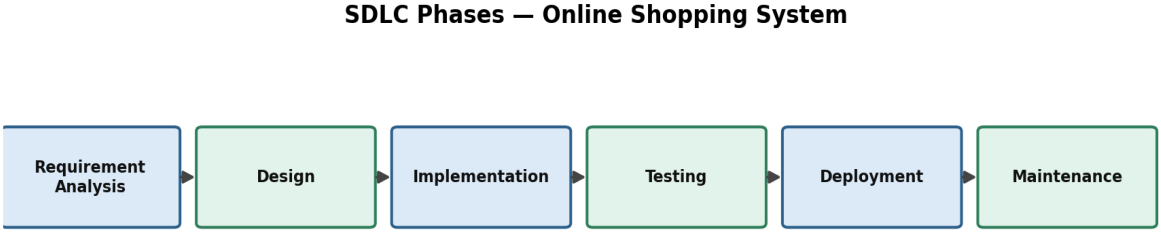


*Fig 3.1 — SDLC phases for the Online Shopping System*

### (b) How each phase contributes to structured development

- Requirement Analysis sets a clear, agreed scope so design and coding are not based on guesswork.
- Design converts requirements into a technical blueprint, ensuring components fit together before coding begins.
- Implementation follows the design systematically, reducing rework and keeping modules consistent.
- Testing validates that the delivered system meets requirements before customers are exposed to it.
- Deployment introduces the system in a controlled, planned manner (e.g., staged rollout).
- Maintenance keeps the system reliable and relevant as usage patterns and business needs evolve.

Together, these phases ensure proper planning and execution at every stage of the project.

### (c) Importance of SDLC in delivering reliable software

- Reduces risk by catching requirement and design errors early, when they are cheapest to fix.
- Improves quality through structured testing before release.
- Provides predictability in cost, schedule and scope through defined milestones.
- Creates documentation and traceability that support long-term maintenance.

## Question 4

## Question 4 — SDLC Models

> *Consider a project where requirements are clearly defined from the beginning.*

### (a) Different SDLC models

- **Waterfall Model:** A linear, sequential model where each phase (requirements → design → implementation → testing → maintenance) must be completed before the next begins. Best suited when requirements are stable and well understood.
- **Iterative Model:** The system is built through repeated cycles (iterations), with each iteration adding or refining functionality based on feedback from the previous cycle.
- **Spiral Model:** Combines iterative development with systematic risk analysis; each loop of the spiral covers objective-setting, risk analysis, engineering, and planning the next iteration — well suited to large, high-risk projects.
- **Agile Model:** An incremental, highly collaborative model that delivers working software in short sprints, with continuous customer feedback and the flexibility to adapt to changing requirements.

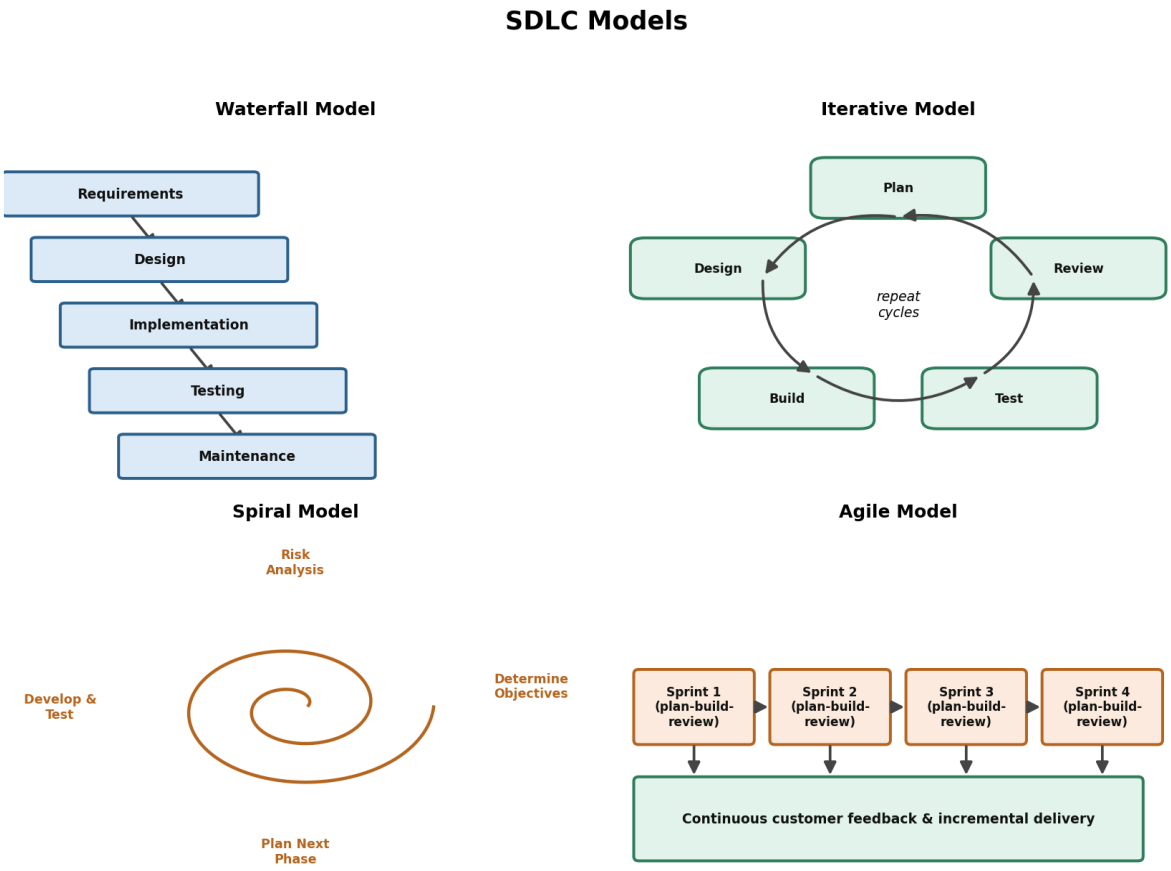


*Fig 4.1 — Waterfall, Iterative, Spiral and Agile models compared*

## (b) Waterfall vs Agile

| Aspect | Waterfall Model | Agile Model |
|---|---|---|
| Approach | Sequential, one phase after another | Iterative and incremental, in short sprints |
| Flexibility | Rigid; changes are costly once a phase is complete | Flexible; adapts easily to changing requirements |
| Customer involvement | Mainly at the start and end | Continuous, throughout development |

| Aspect | Waterfall Model | Agile Model |
|---|---|---|
| Delivery | Single final delivery at project end | Frequent incremental releases |
| Best suited for | Projects with clear, stable requirements | Projects with evolving or unclear requirements |

### (c) Suitable model for dynamic requirements

The Agile model is the most suitable choice when requirements are dynamic and likely to change during development, because it is built around short, iterative sprints and continuous stakeholder feedback. This allows the team to adjust priorities, incorporate new requirements, and course-correct quickly — something the rigid, sequential structure of the Waterfall model cannot accommodate without expensive rework.

## Question 5

## Question 5 — ATM System (UML)

> *Suppose a system analyst is designing an ATM System.*

### (a) Importance of UML in OOAD

The Unified Modeling Language (UML) provides a standard, visual way to represent the structure and behaviour of a software system. It allows analysts, designers, developers and clients to communicate a shared understanding of the system before a single line of code is written, making complex object-oriented designs (such as an ATM's classes, interactions, and workflows) easier to plan, review, and validate.

### (b) Common UML diagrams

- **Use Case Diagram:** Shows the functional requirements of the system from the user's perspective — the actors (Customer, Bank Server) and the actions they can perform (Check Balance, Withdraw Cash, Deposit Cash, Transfer Funds).

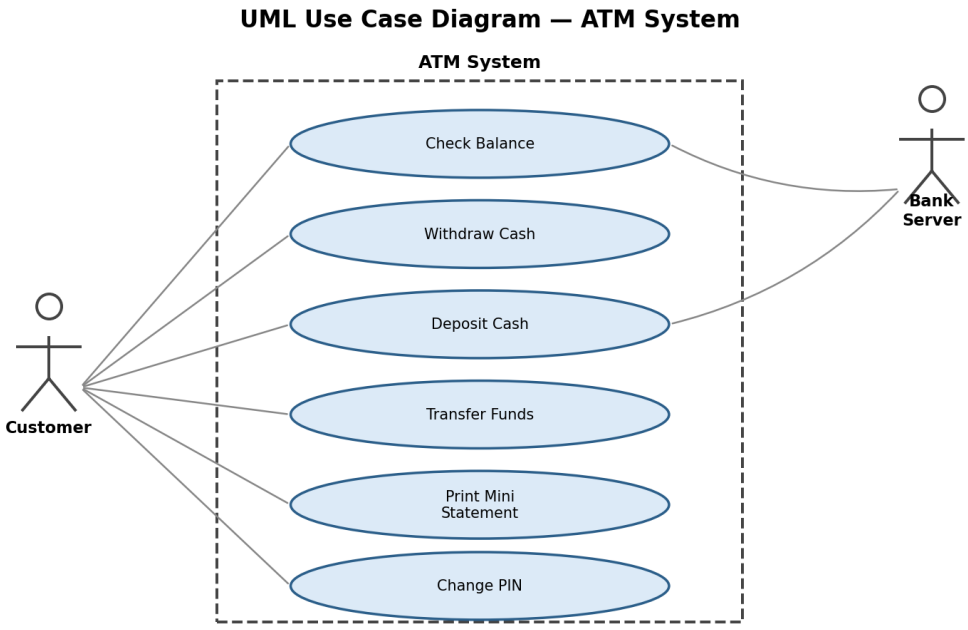


*Fig 5.1 — Use Case Diagram for the ATM System*

- **Class Diagram:** Shows the static structure of the system — the classes (ATM, Account, Card, Transaction), their attributes, methods, and the relationships/multiplicities between them.

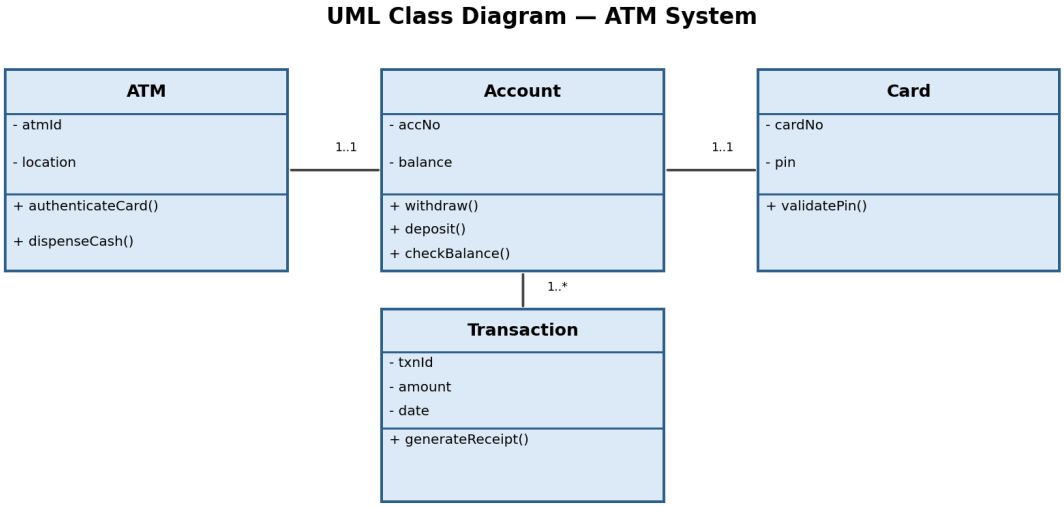


*Fig 5.2 — Class Diagram for the ATM System*

- **Sequence Diagram:** Shows the time-ordered sequence of messages exchanged between objects to complete a specific scenario, such as a cash withdrawal, from card insertion to cash dispensing.

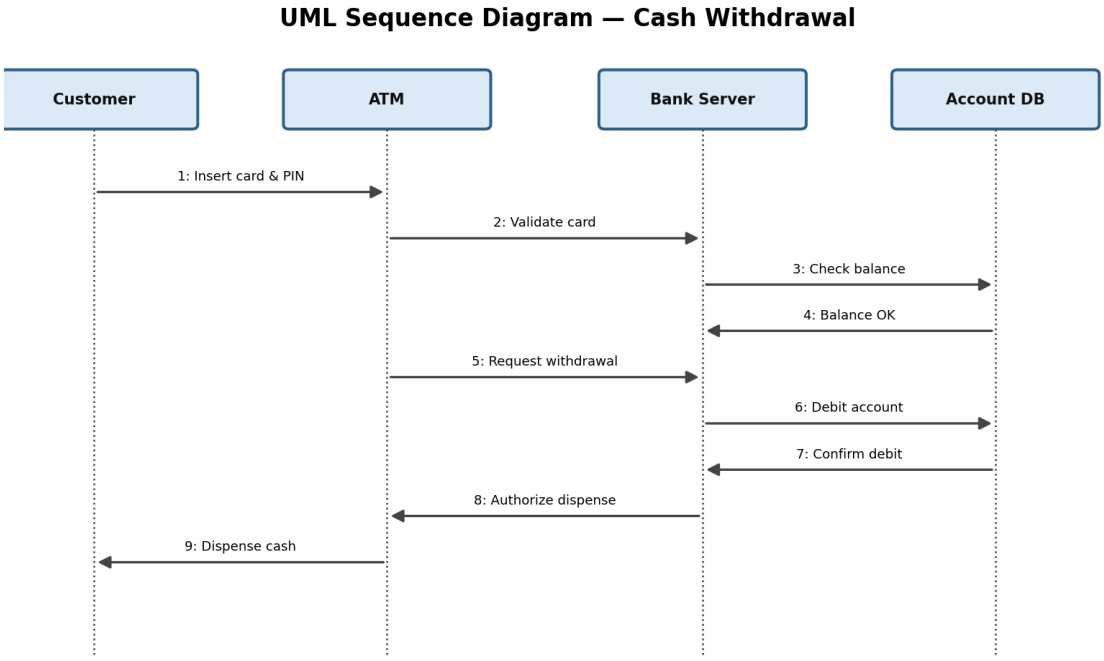


*Fig 5.3 — Sequence Diagram for a Cash Withdrawal*

- **Activity Diagram:** Shows the flow of control and decision logic within a process — such as validating a PIN, checking balance, and dispensing cash or displaying an error.

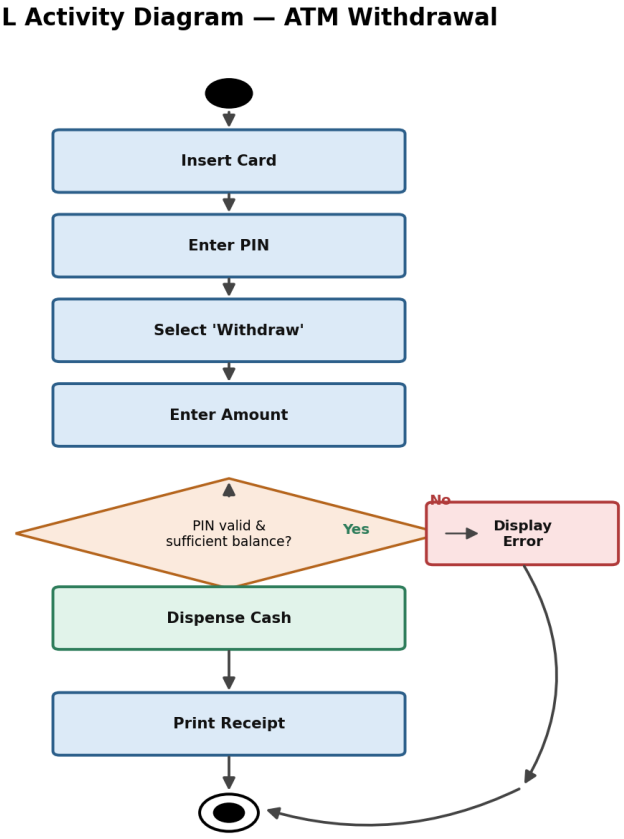


*Fig 5.4 — Activity Diagram for ATM Withdrawal*

### (c) How UML supports object-oriented methodologies

- Provides a common visual language for analysis, design, and communication among all project stakeholders.
- Captures both static structure (class diagrams) and dynamic behaviour (sequence, activity diagrams), covering the full picture of an object-oriented system.
- Helps identify classes, objects, and their relationships early, directly supporting object-oriented analysis and design.
- Serves as documentation that eases future maintenance and onboarding of new developers.

## Question 6

## Question 6 — Hospital Management System

> *Consider a scenario where a system needs to be designed for Hospital Management.*

### (a) Structured design approach

The structured design approach breaks a software system down into a hierarchy of functions and procedures, following a top-down decomposition. The system is organised around what it does (processes), with data typically stored separately and passed between functions as parameters or shared global structures. In a Hospital Management System, this would mean separate functions such as registerPatient(), scheduleAppointment(), and generateBill(), all operating on shared patient and appointment data.

## (b) Structured vs Object-Oriented approach

| Aspect | Structured Approach | Object-Oriented Approach |
|---|---|---|
| Organising principle | Function-based (what the system does) | Object-based (real-world entities modelled as objects) |
| Data handling | Data and functions are largely separate | Data and behaviour are encapsulated together in objects |
| Reusability | Limited; functions are often context-specific | High; classes and objects can be reused/extended |
| Maintainability | Changes can ripple across many functions sharing data | Changes are localised within well-defined classes |
| Example | registerPatient(), generateBill() as standalone functions | Patient, Appointment, Billing as classes with their own methods |

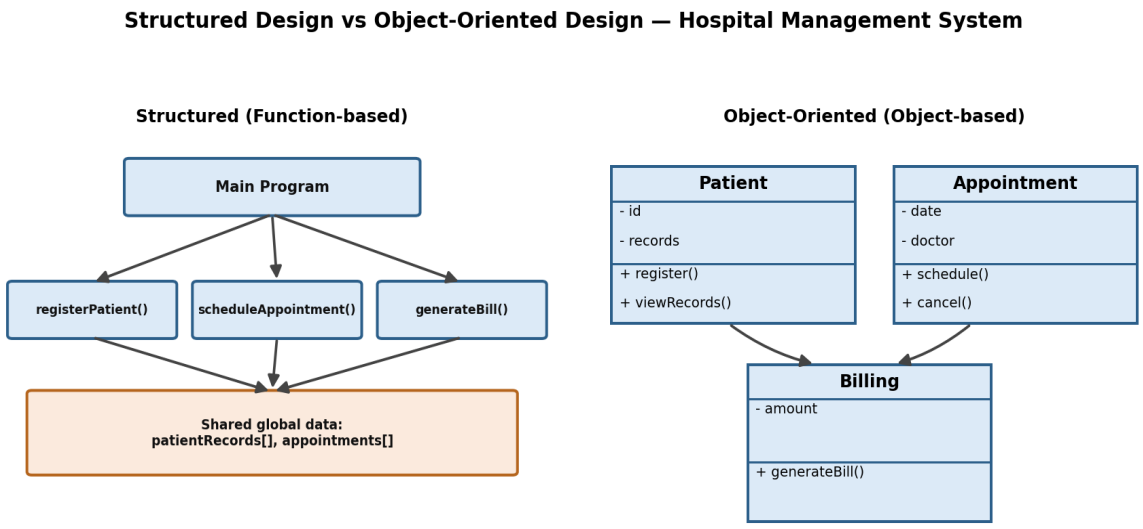


*Fig 6.1 — Structured vs Object-Oriented design for Hospital Management*

## (c) How modular design improves efficiency and maintenance

- Easy debugging: issues can be isolated to a specific module (e.g., Billing) without affecting others.
- Scalability: new modules (e.g., a Pharmacy module) can be added without disturbing existing ones.
- Reuse: well-defined modules such as Patient or Appointment can be reused across other parts of the hospital system, or even other projects.
- Parallel development: independent teams can work on separate modules simultaneously, speeding up delivery.

## Rubric Reference (for self-check)

Each sub-question is assessed against the following criteria — ensure answers are conceptually accurate, clearly explained with examples, and well organised.

| Criteria | Weightage | What Level 4 (Exemplary) looks like |
|---|---|---|
| Understanding of Concepts | 25% | Thorough, accurate understanding of concepts |
| Explanation | 25% | Detailed, well-explained answers with relevant examples |
| Application | 20% | Effectively applies concepts with strong analytical ability |
| Organization | 15% | Well-structured answers with logical flow and coherence |
| Accuracy | 10% | Completely accurate and covers all required points |